

Documentação Técnica Final - Sistema VidaPlena (Etapa AV2)

1. Módulo de Pessoas e Convênios

1.1 Jornadas de Usuário

Jornada 1: Cadastro Simplificado de Paciente

Viabiliza o registro ágil na recepção contendo apenas as informações essenciais (Nome e CPF). O sistema aciona rotinas de encapsulamento para bloquear parâmetros nulos ou vazios no momento da instanciação.

Jornada 2: Cadastro Completo e Controle de Duplicidade

Registra o paciente de forma abrangente com dados de contato e vinculação com planos de saúde (Associação). O sistema intercepta entradas inválidas (ex: idades negativas) protegendo a consistência dos dados.

Jornada 27: Busca Eficiente de Pacientes

Recupera instantaneamente o histórico do paciente por chave de CPF. Através de ligação dinâmica, o sistema resolve e invoca os métodos especializados de resumo em tempo de execução.

1.2 Diagrama de Classes UML

Pessoa <i><<abstract>></i>
nome: String # cpf: String # idade: int # telefone: String
+ Pessoa(nome, cpf) + Pessoa(nome, cpf, idade, tel) + getNome(): String

Pessoa
<<abstract>>
+ setNome(nome): void + getCpf(): String + setCpf(cpf): void + getIdade(): int + setIdade(idade): void + getTelefone(): String + setTelefone(tel): void + possuiCpfPreenchido(): boolean + exibirResumo(): void <<abstract>>

Paciente
- convenio: Convenio - ativo: boolean
+ Paciente(nome, cpf) + Paciente(nome, cpf, idade, tel) + Paciente(nome, cpf, idade, tel, convenio) + getConvenio(): Convenio + setConvenio(convenio): void + isAtivo(): boolean + setAtivo(ativo): void + setCpf(cpf): void + setIdade(idade): void - setAvisoValidarCpf(cpf): void + complementar(idade, tel): void + complementar(idade, tel, convenio): void + desativar(): void + exibirResumo(): void

Convenio
- nome: String - cnpj: String - percentualCobertura: double - especialidadesCobertas: Set<String>
+ Convenio(nome, cnpj, percentualCobertura) + Convenio(nome, cnpj, pct, especialidades) + getNome(): String + setNome(nome): void + getCnpj(): String + setCnpj(cnpj): void + getPercentualCobertura(): double + setPercentualCobertura(pct): void + getEspecialidadesCobertas(): Set<String> + adicionarEspecialidade(especialidade): void + cobreEspecialidade(especialidade): boolean + obterDadosFormatados(): String

2. Módulo de Corpo Técnico e Profissionais

2.1 Jornadas de Usuário

Jornadas 3 e 4: Cadastro de Especialidades (Simples e Múltiplas)

Responsável pela entrada de profissionais que atuam em uma ou diversas frentes. O sistema garante a instância correta das subclasses especializadas, consolidando a flexibilidade técnica da clínica por meio de herança profunda.

Jornada 29: Gestão de Horários Disponíveis

Operacionaliza a alocação e o controle de agendas médicas. Implementa a regra de agregação, permitindo o reaproveitamento de blocos de horários para a otimização da escala de trabalho.

2.2 Diagrama de Classes UML

Profissional <i><<abstract>> (extends Pessoa)</i>
registroProfissional: String # especialidade: String # valorConsulta: double # agenda: List<HorarioDisponivel>
+ Profissional(nome, cpf, registro, espec, valor) + getRegistroProfissional(): String + setRegistroProfissional(registro): void + adicionarHorario(horario: HorarioDisponivel): void # registrarEspecifico(atendimento): void <i><<abstract>></i> + exibirResumo(): void <i><<abstract>></i>

HorarioDisponivel
- diaSemana: String - hora: String - ocupado: boolean
+ HorarioDisponivel(diaSemana, hora) + isOcupado(): boolean + setOcupado(ocupado): void

Fisioterapeuta <i>(extends Profissional)</i>
- totalSessoesPrevistas: int
+ Fisioterapeuta(nome, cpf, registro, espec, valor, sessoes) + getTotalSessoesPrevistas(): int # registrarEspecifico(atendimento): void

Fisioterapeuta <i>(extends Profissional)</i>
+ exibirResumo(): void

3. Módulo de Agendamentos e Prontuários

3.1 Jornadas de Usuário

Jornadas 5 e 6: Gestão de Consultas (Regular e Retorno)

Gerencia a criação, cancelamento e remarcação de consultas. O sistema implementa construtores sobrecarregados para diferenciar consultas regulares de retornos, validando os contratos das interfaces transversais da aplicação.

Jornadas 7 e 8: Registro de Atendimento e Prontuário Clínico

Ao iniciar um atendimento, a classe Atendimento instancia restritamente um objeto Prontuario (relação de Composição forte), garantindo que não existam prontuários órfãos. Permite o armazenamento de observações, diagnóstico e procedimentos.

3.2 Diagrama de Classes UML

Consulta <i><<implements Agendavel, Exportavel>></i>
- pacienteCpf: String - profissionalNome: String - dataHora: String - tipo: String
+ Consulta(cpf, prof, dataHora) + Consulta(cpf, prof, dataHora, tipo) + agendar(): void + cancelar(): void + remarcar(): void + exportarDados(): String

Atendimento
<<implements Exportavel>>
- consulta: Consulta - prontuario: Prontuario
+ Atendimento(consulta) + registrarClinico(obs, diag): void + exportarDados(): String

Prontuario
- diagnostico: String - observacoes: String - procedimentos: List<String>
~ Prontuario() <<package-private>> + setDiagnostico(diag): void + setObservacoes(obs): void + adicionarProcedimento(proc): void

4. Módulo de Faturamento e Finanças

4.1 Jornadas de Usuário

Jornada 9: Fluxo de Dinheiro / Pix

Processamento de transações à vista. O motor polimórfico aplica uma dedução fixa de 5% de desconto sobre o valor base do procedimento.

Jornada 10: Fluxo de Cartão (Crédito/Débito)

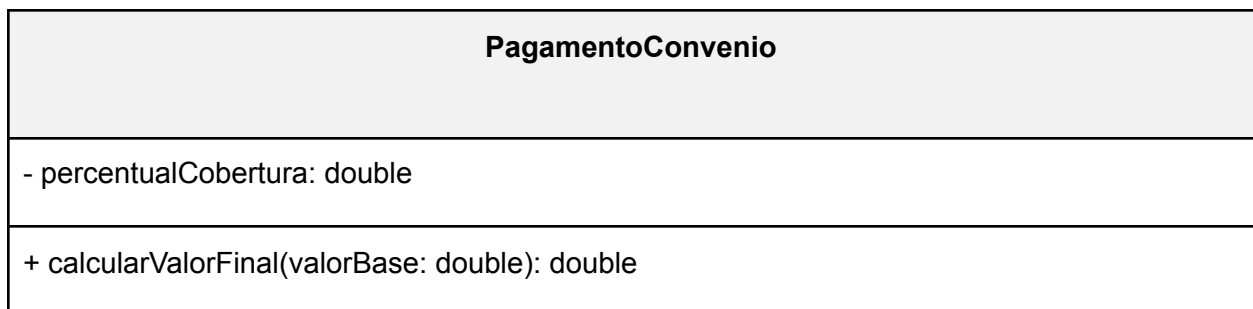
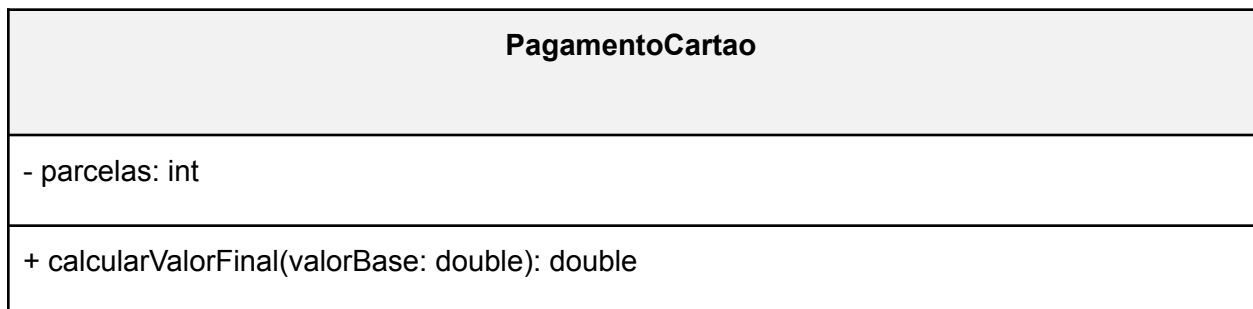
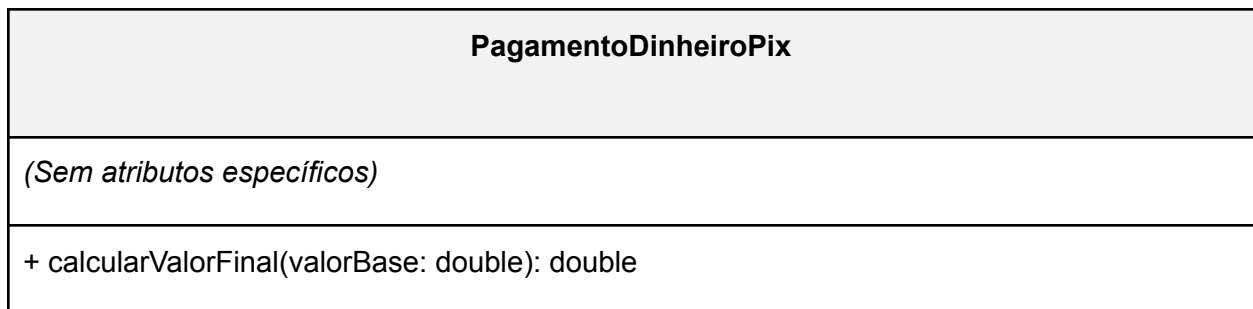
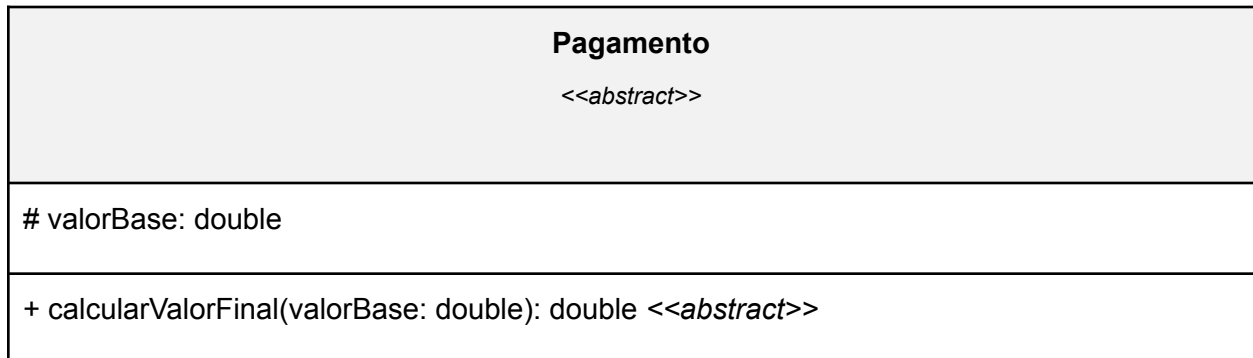
Gerencia pagamentos via adquirentes, permitindo parcelamento em até 3 vezes sem acréscimo. Quantidades superiores recebem taxas de juros incrementais.

Jornada 11: Fluxo de Convênio Saúde

Integração com planos de saúde, realizando o abatimento baseado no percentual de cobertura

contratado e calculando a coparticipação residual do paciente.

4.2 Diagrama de Classes UML



5. Módulo de Interfaces e Relatórios Transversais

5.1 Jornadas de Usuário

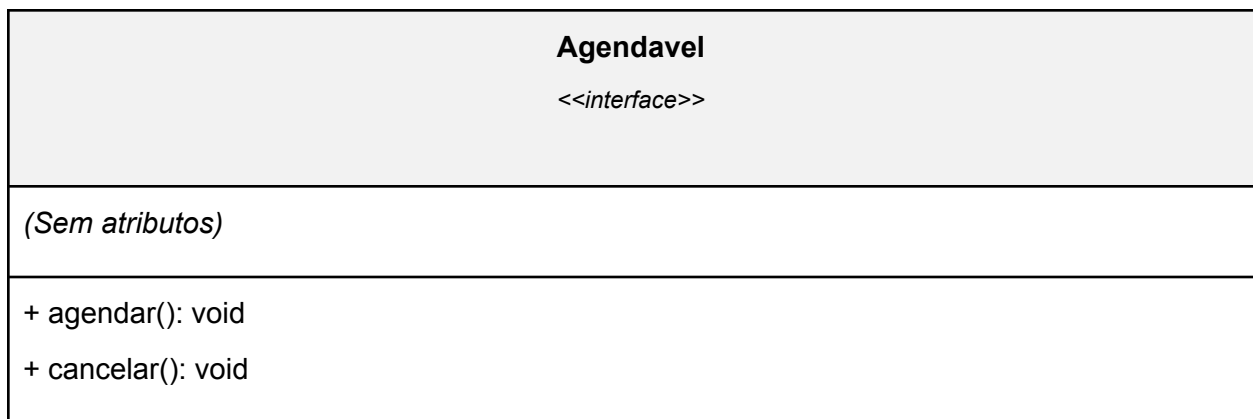
Jornada 12: Relatório de Consultas por Profissional

Permite a consolidação e exibição de todas as consultas associadas a um profissional de saúde específico. O sistema varre a coleção de consultas filtrando pela identificação do profissional e exibe em tela uma lista ordenada com os dados da agenda.

Jornada 13: Relatório de Faturamento por Período

Calcula e demonstra o somatório financeiro exato recebido ou a receber pela clínica dentro de um intervalo de tempo delimitado. O sistema localiza a lista polimórfica de pagamentos e invoca o método `calcularValorFinal()`, consolidando a somatória geral das taxas para exibir o faturamento total.

5.2 Diagrama de Classes UML (Contratos)



6. Módulo de Exceções Personalizadas

6.1 Jornadas de Usuário

Jornada 30: Tratamento Completo de Operações Inválidas

Intercepta de maneira preemptiva anomalias operacionais e quebras de regras de negócio.

Utiliza exceções verificadas (herdadas de `java.lang.Exception`) com construtores sobrecarregados para capturar a causa raiz, garantindo que o operador receba feedback seguro sem corromper o estado do sistema.

6.2 Estrutura de Exceções Negociais

Classe de Exceção	Gatilho de Negócio
PacienteInativoException	Disparada ao tentar agendar consulta para um paciente desativado.
HorarioIndisponivelException	Disparada quando o horário informado já está ocupado ou fora da jornada do profissional.
ConvenioNaoCobreException	Disparada caso a especialidade requisitada não conste na cobertura do convênio do paciente.
ConsultaInvalidaException	Inconsistências temporais, como agendar retorno fora do prazo limite.
LimiteParcelasExcedidoException	Disparada ao registrar faturamento parcelado acima do teto permitido comercialmente.
PacienteNaoEncontradoException	Falha ao tentar buscar CPF inexistente no mapa indexado.
OperacaoInvalidaException	Impede ações ilegais como cancelar consulta já cancelada ou realizada.

7. Módulo de Serviços e Integração

7.1 Jornadas de Usuário

Jornada 28: Controle de Unicidade de CPF

Garante em tempo real que o sistema não permita cadastros duplicados. A estrutura `HashSet`

impede a duplicidade nativamente e garante verificação em tempo constante $O(1)$.

Orquestração Central e Resiliência

A classe Main atua como escudo da aplicação, envolvendo chamadas de serviço em blocos try/catch/finally para mapeamento individualizado de erros (ex: InputMismatchException).

Garante também a execução ininterrupta dos relatórios polimórficos.

7.2 Diagrama de Classes UML

